returns a stream of results. For example, inside the chain, the `fetchWeatherData` function is called to fetch the weather data for the given city. The language model generates a response based on the processed weather data.

3. The `createStreamableUI` function is called to create a streamable UI component. This component will be used to update the UI with real-time information.
4. The streamable UI and the message history are returned, providing the user with the generated response and the updated UI.

**Project: chat-rsc-langchain-chains**

The code for this example project is in the ch06/chat-rsc-langchain-chains folder. You can use that example by running this command in the root folder:

```
$ npm run dev -w ch06/chat-rsc-langchain-chains
// .env

OPENAI_API_KEY=<SECRET_KEY>
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
```

For details on setting up and creating API keys, refer to the appendix.

Listing 6.4 shows the new changes we must add to the actions.jsx file that implements the functionality for asking the LLM for a weather report of a particular location. The LLM will respond in a particular format to the user. It's worth mentioning that the rest of the code stays the same without modifications. To see this in action, launch your Astra AI application and try entering a specific weather prompt like "What's the weather like in Galway, Ireland?" Astra AI will process your request and respond with a detailed weather report for the specified location, demonstrating its new capability to provide structured information.

**Listing 6.4 Integrating the chain into the conversation chain**

```
export async function getChainStream(city) {
  const chain = RunnableLambda.from(fetchWeatherData)
                              .pipe(promptTemplate)
                              .pipe(llm)
                              .pipe(new StringOutputParser());

  const stream = await chain.stream({ city });
  return stream;
}

export async function continueConversation(input) {
  const aiState = getMutableAIState();
  const stream = createStreamableUI();

  stream.update(<div>Processing your request...</div>);
```

**Builds a chain from weather data → prompt → LLM → parser**

**Starts streaming results from the chain**

**Creates a live UI stream for updates**